UTS
UNIVERSITY OF TECHNOLOGY SYDNEY

Machine Learning Algorithms and Applications

Lecture 2

(Block 1)

UPASS

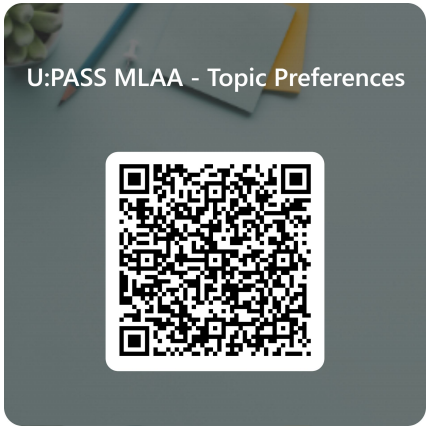
UPASS Sessions (starting next week)
with **Katherin Gomez Londono**

on

- **Thursdays 4-5pm CB01.05.01**
- **Fridays 4-5pm CB.08.03.005**

www.uts.edu.au/upass-sessions

You can mention what topics you want to cover next sessions by filling this form:



Plan for Today

Regression Metrics

Regularisation

Dataset Splitting

Lab

3

Regression Metrics

Mean Squared Error

Formula:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Y_i actual target value for observation i

$\hat{Y}_i$ predicted target value for observation i

i index of observation at position i

n total number of observations

⇨ $Y_i - \hat{Y}_i$ Error for observation i

⇨ $(Y_i - \hat{Y}_i)^2$ Squared error observation i

⇨ $\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$ Sum of squared errors

⇨ $\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$ Mean of squared errors

MSE = 0 (Perfect score)

Higher MSE means worse predictions

Mean Squared Error

xi	Y_i	$\hat{Y}_i$
1	2	4
2	4	6
3	6	8
4	8	10

$Y_i - \hat{Y}_i$	$(Y_i - \hat{Y}_i)^2$
2 - 4 = -2	(-2)^2 = 4
4 - 6 = -2	(-2)^2 = 4
6 - 8 = -2	(-2)^2 = 4
10 - 8 = -2	(-2)^2 = 4

$\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$
4+4+4+4=16

$\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$
16/4 = 4

sklearn:

```
from sklearn.metrics import mean_squared_error
y_true = [2, 4, 6, 8]
y_pred = [4, 6, 8, 10]
mean_squared_error(y_true, y_pred)
```

```
4.0
```

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html

Note:
Root Mean Squared Error:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2}$$

Mean Absolute Error

Formula:

$$\mathrm{MAE} = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j|$$

Y_i actual target value for observation i

$\hat{Y}_i$ predicted target value for observation i

i index of observation at position i

n total number of observations

⇨ $y_j - \hat{y}_j$ Error for observation i

⇨ $|y_j - \hat{y}_j|$ Absolute error observation i

⇨ $\sum_{j=1}^{n}|y_j - \hat{y}_j|$ Sum of absolute errors

⇨ $\frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j|$ Mean of absolute errors

MAE = 0 (Perfect score)

Higher MAE means worse predictions

Mean Absolute Error

xi	Y_i	$\hat{Y}_i$
1	2	4
2	4	6
3	6	8
4	8	10

$y_j - \hat{y}_j$	$\lvert y_j - \hat{y}_j \rvert$
2 - 4 = -2	|-2| = 2
4 - 6 = -2	|-2| = 2
6 - 8 = -2	|-2| = 2
10 - 8 = -2	|-2| = 2

$\sum_{j=1}^{n} \lvert y_j - \hat{y}_j \rvert$
2+2+2+2=8

$\frac{1}{n} \sum_{j=1}^{n} \lvert y_j - \hat{y}_j \rvert$
8/4 = 2

sklearn:

```
from sklearn.metrics import mean_absolute_error
y_true = [2, 4, 6, 8]
y_pred = [4, 6, 8, 10]
mean_absolute_error(y_true, y_pred)
```

```
2.0
```

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html

MSE vs MAE

CASE 1: Evenly distributed errors

ID	Error	|Error|	Error^2
1	2	2	4
2	2	2	4
3	2	2	4
4	2	2	4
5	2	2	4
6	2	2	4
7	2	2	4
8	2	2	4
9	2	2	4
10	2	2	4

MAE	RMSE
2.000	2.000

CASE 2: Small variance in errors

ID	Error	|Error|	Error^2
1	1	1	1
2	1	1	1
3	1	1	1
4	1	1	1
5	1	1	1
6	3	3	9
7	3	3	9
8	3	3	9
9	3	3	9
10	3	3	9

MAE	RMSE
2.000	2.236

CASE 3: Large error outlier

ID	Error	|Error|	Error^2
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0
5	0	0	0
6	0	0	0
7	0	0	0
8	0	0	0
9	0	0	0
10	20	20	400

MAE	RMSE
2.000	6.325

https://medium.com/human-in-a-machine-world/mae-and-rmse-which-metric-is-better-e60ac3bde13d

R square

Formula:

$$\hat{R}^2 = 1 - \frac{\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}$$

- Y_i actual target value for observation i
- $\hat{Y}_i$ predicted target value for observation i
- $\bar{Y}$ Expected value for target (ie overall mean)
- i index of observation at position i
- n total number of observations

⇨ $\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$ Sum of squared errors (for model predictions)

⇨ $\sum_{i=1}^{n}(Y_i - \bar{Y})^2$ Sum of squared errors (for model always predicting the mean)

⇨ $\frac{\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}$ Comparison of a model against a not-so random model (always predicting mean)

Used for Goodness of Fit (ie how much the variance of the target is explained by the feature for your model)
=> Analyse the impact of your features (not for assessing the performance of a model)

A model that disregards input features will have a score of 0

R square

xi	Y_i	$\hat{Y}_i$	$(Y_i - \bar{Y})^2$	$(Y_i - \hat{Y}_i)^2$
1	2	4	(2-5)^2= 9	(2-4)^2 = 4
2	4	4	(4-5)^2 = 1	(4-4)^2 = 0
3	6	4	(6-5)^2= 1	(6-4)^2 = 4
4	8	4	(8-5)^2 = 9	(8-4)^2 = 16

f(x) = 4 + 0 * x = 4

$\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$
4+0+4+16=24
$\sum_{i=1}^{n}(Y_i - \bar{Y})^2$
9+1+1+9=20

$\hat{R}^2 = 1 - \frac{\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}$
1 - (24/20) = -0.2

Model doesn't depend on x => low R square

Note: The total error from this model is higher than the one always predicting the mean

R square

xi	Y_i	$\hat{Y}_i$	$(Y_i - \bar{Y})^2$	$(Y_i - \hat{Y}_i)^2$
1	2	4	(2-5)^2= 9	(2-4)^2 = 4
2	4	6	(4-5)^2 = 1	(4-6)^2 = 4
3	6	8	(6-5)^2= 1	(6-8)^2 = 4
4	8	10	(8-5)^2 = 9	(8-10)^2 = 4

f(x) = 2 + 2 * x

$\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$
4+4+4+4=16
$\sum_{i=1}^{n}(Y_i - \bar{Y})^2$
9+1+1+9=20

$\hat{R}^2 = 1 - \frac{\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}$
1 - (16/20) = 0.2

Model doesn't depend a lot on x => low R square

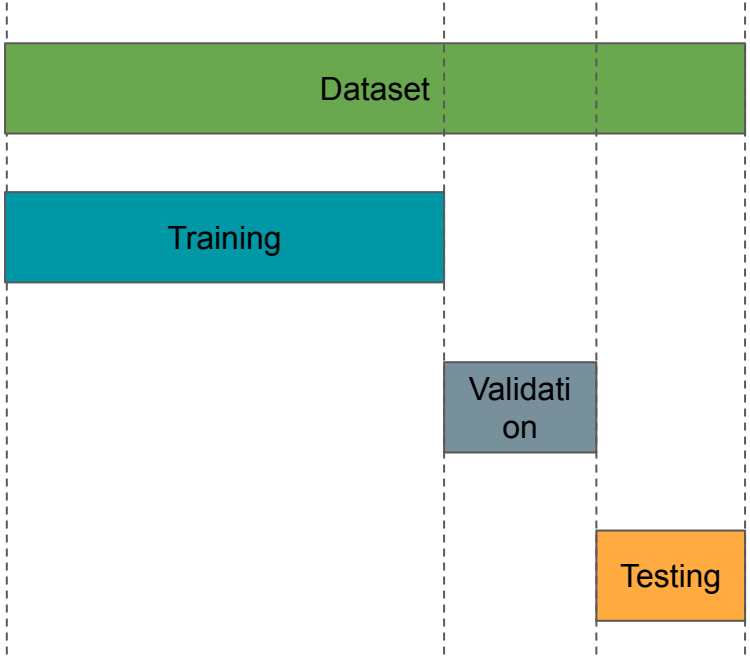
Note: The total error from this model is lower than the one always predicting the mean. This model is making less error and is quite close to the perfect solution but R square still penalises it

Perfect solution:

f(x) = 2 * x

Dataset Splitting

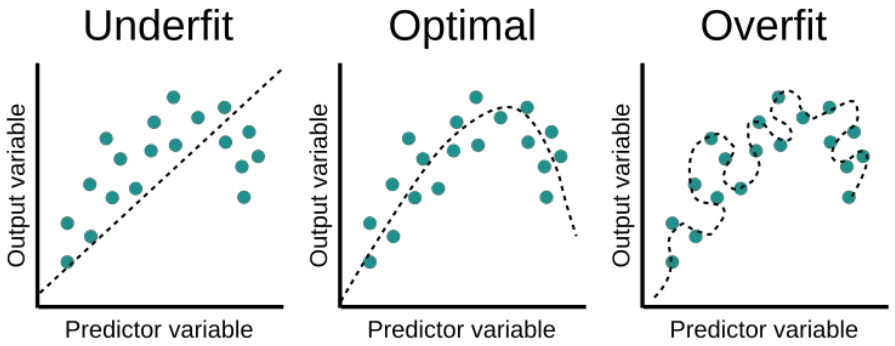
Different Sets

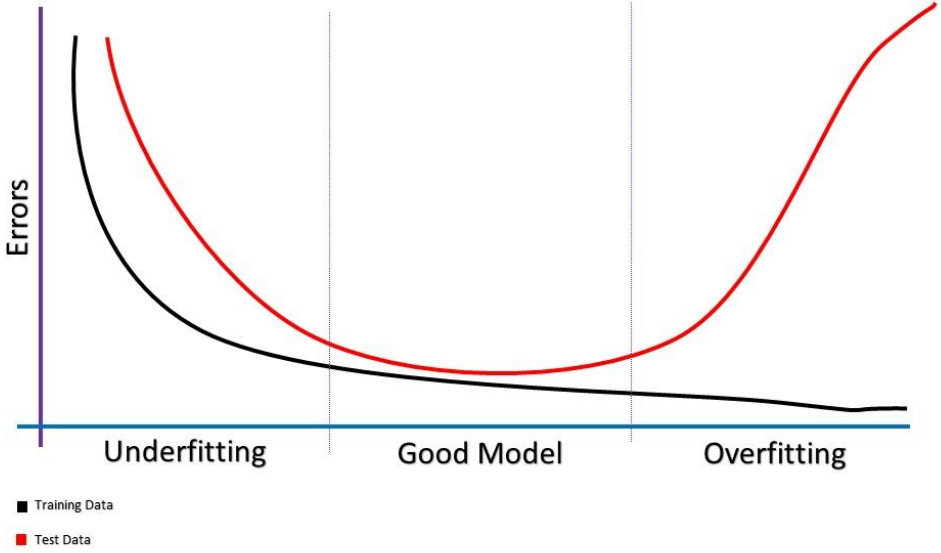


- Original dataset
- Subset of the original dataset used for training a model (parameters learning)
- Subset of the original dataset used for evaluating model performance and hyperparameter tuning
- Subset of the original dataset used for assessing model performance and generalisation capability

Underfitting vs Overfitting

(Bias-Variance tradeoff)



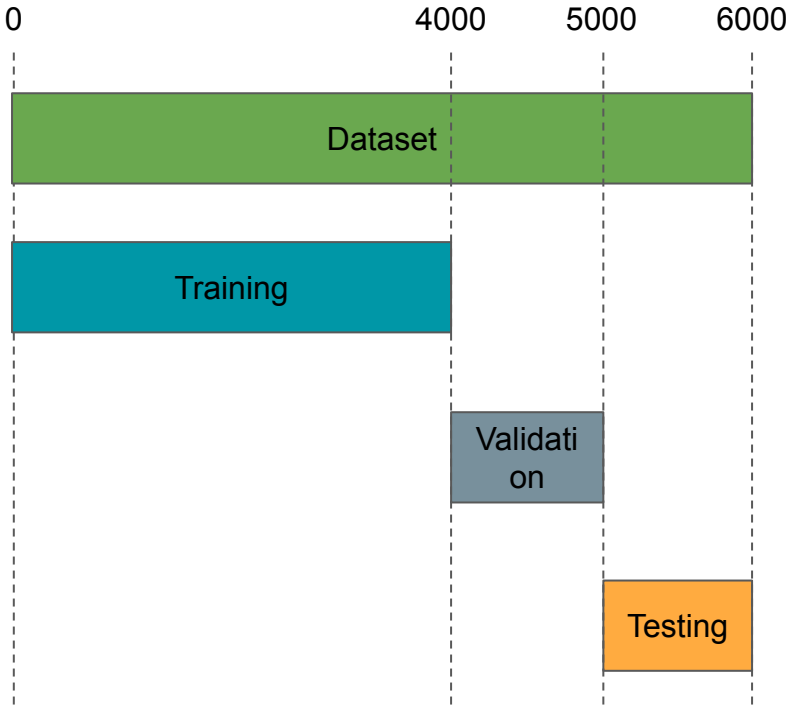


Underfitting
Model performance is quite low. High errors on training set

Optimal
Model is performing well on both training and test sets

Overfitting
Model is performing well on training set but not on testing set

Splitting Strategy – Index

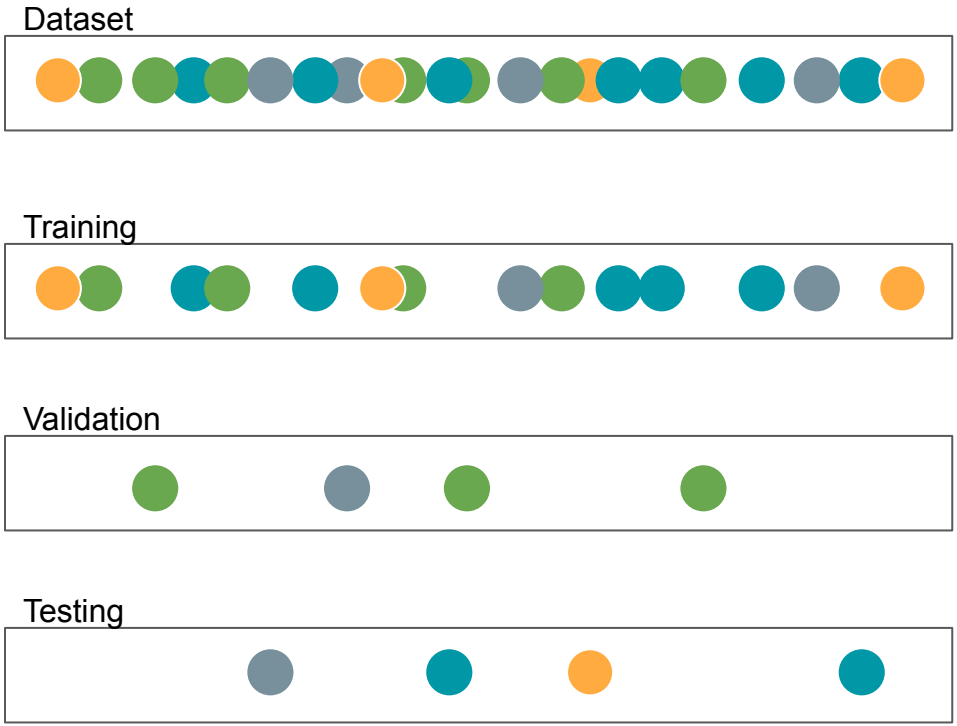



```
train_data = df[:4000]
```

```
validation_data = df[4000:5000]
```

```
test_data = df[5000:]
```

Splitting Strategy – Random



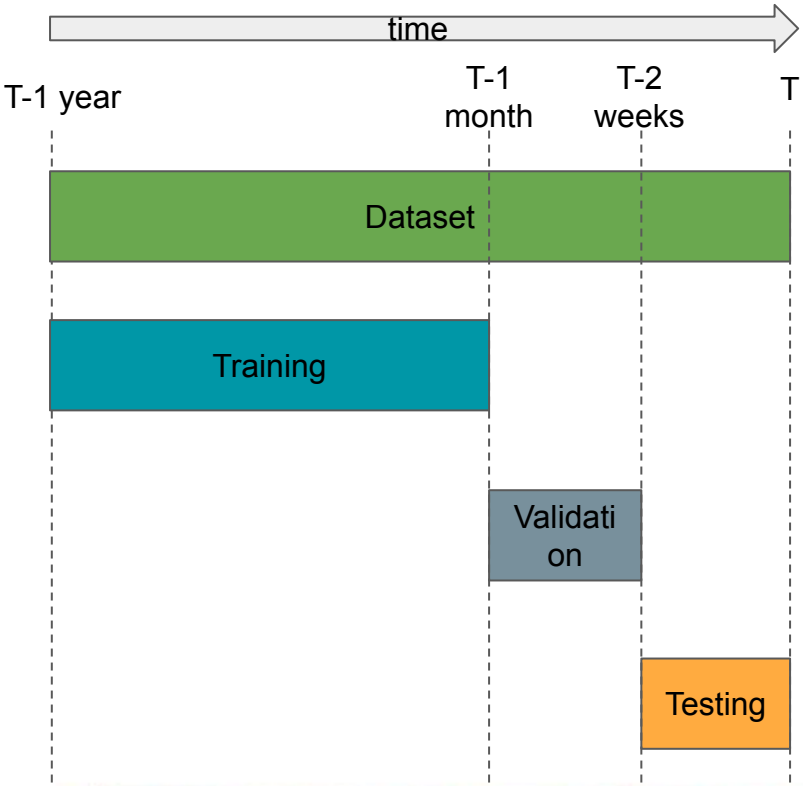
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

```
from sklearn.model_selection import train_test_split
```

```
X_data, X_test, y_data, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
```

```
X_train, X_val, y_train, y_val = train_test_split(X_data, y_data,
    test_size=0.2, random_state=42)
```

Splitting Strategy – Time




```
train_mask = df[time_col] < '2020-08-01'
train_data = df[mask]
```

```
val_mask_low = (df[time_col] >= '2020-08-01')
val_mask_high = (df[time_col] < '2020-08-15')
validation_data = df[val_mask_low & val_mask_high]
```

```
test_mask_low = (df[time_col] >= '2020-08-15')
test_mask_high = (df[time_col] < '2020-09-01')
test_data = df[test_mask_low & test_mask_high]
```

Why is it important to split data?

Training

In order to help the model learns you need to provide some examples

Performance

After training, you want to assess how well your model is performing on new data

Tuning

You will usually train multiple models with different set on hyperparameters

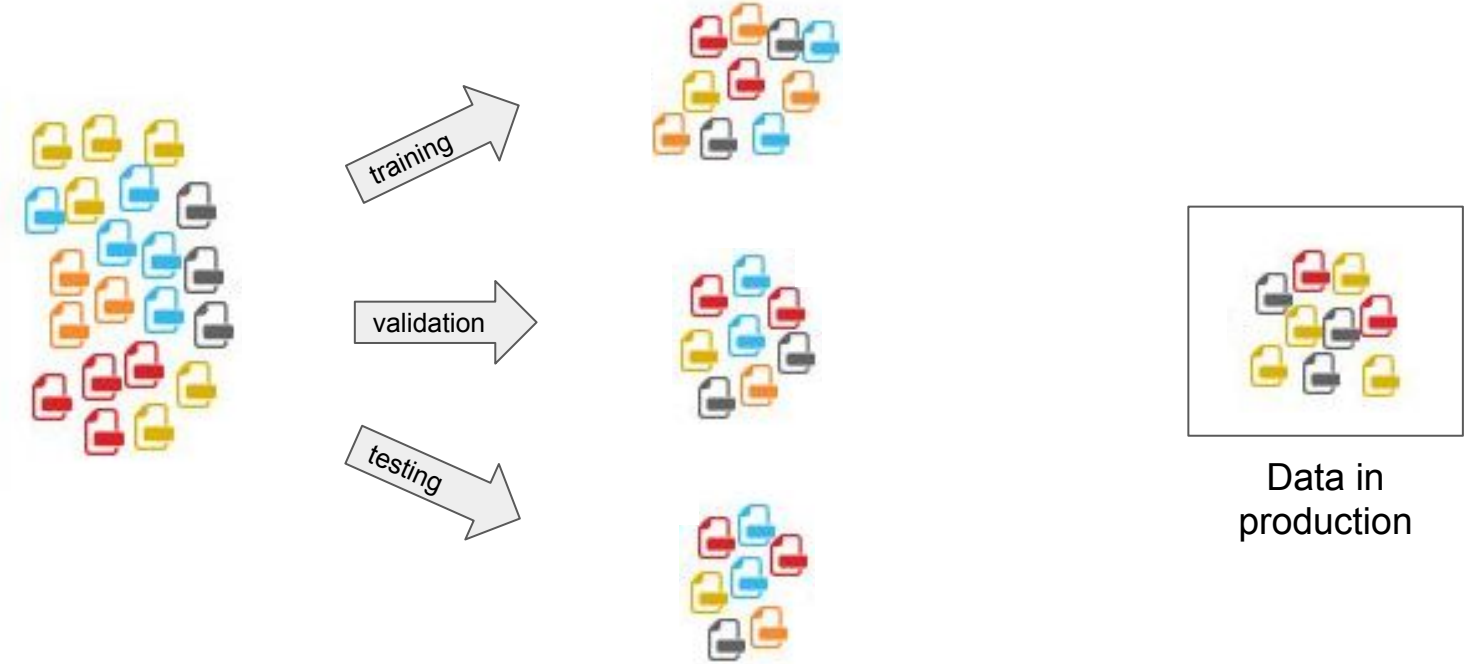
Generalizability

The goal is not just to predict accurately but being able to do it on unseen data (model in production)

Assessing your model properly is more important that choosing which algorithm to train

19

Splitting Mistake



Regularisation

Regularisation

Mechanism applied to a model for reducing its level of complexity.

Adding penalties to model parameters that will shrink them towards zero.

Used for reducing overfitting of a model

$$\hat{y} = f(X) = \sum_{i=0}^{n} w_i * x_i$$

$$\hat{y} = f(X) = w_0 + w_1 * x_1 + w_2 * x_2 \ + \ldots \ + w_{n-1} * x_{n-1} + w_n * x_n$$

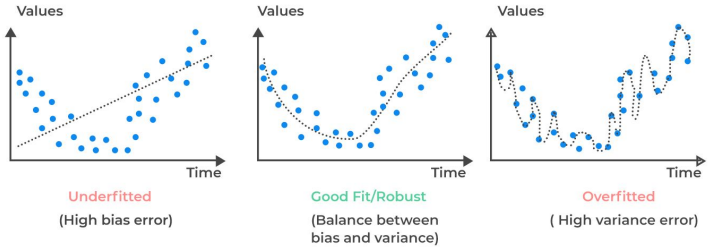
If n=10000, you can have up to 10000 non-zero coefficients
=> complex model

Some of these coefficients may have small value and are only used for very rare specific cases
=> overfitting

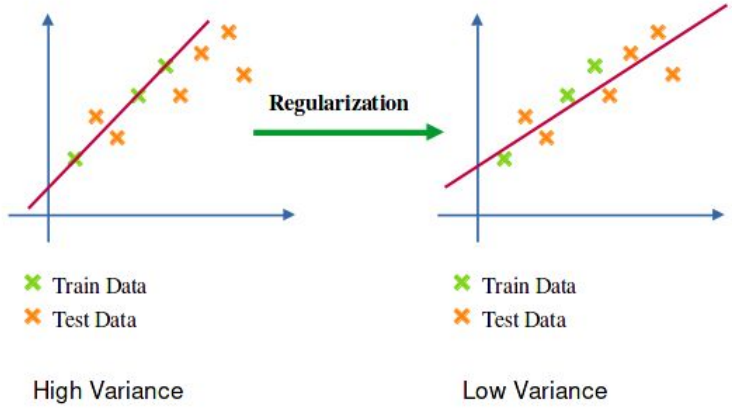
Goal: Remove these small coefficients by adding penalties (regularisation)

Regularisation

Regularisation is used for “simplifying” linear regression models and avoid overfitting



It works by adding penalties in the loss function => Reduce weights



Loss Function and Penalty

ML models learn relevant patterns from data by using a loss function to minimise their prediction errors

$$Loss = Error(y, \hat{y}) \Longrightarrow minimise(Loss)$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

We can a penalty term to the model objective that is a function of the weights:

$$minimise(Loss + Penalty) = Minimise(Error(y, \hat{y}) + Penalty)$$

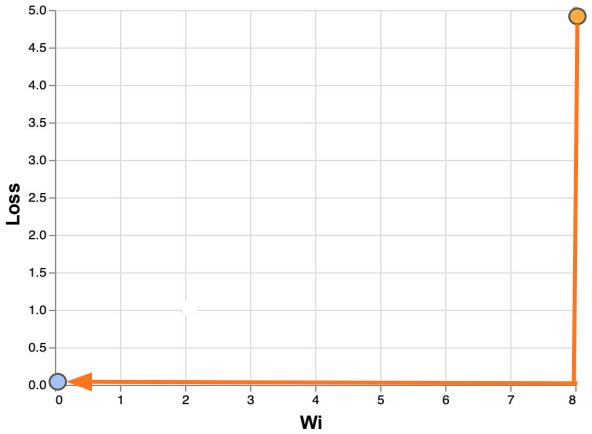
Minimise difference between predictions and true value

Minimise weights value

L1 Regularisation

The penalty component of L1 Regularisation is the absolute value of the weights.

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} |w_i|$$



L1 Regularisation tries to bring down to 0 not useful features. It can be used for feature selection purpose.

Lasso Regression

Lasso Regression is the linear model that implements L1 Regularisation

```
from sklearn.linear_model import Lasso

lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
```

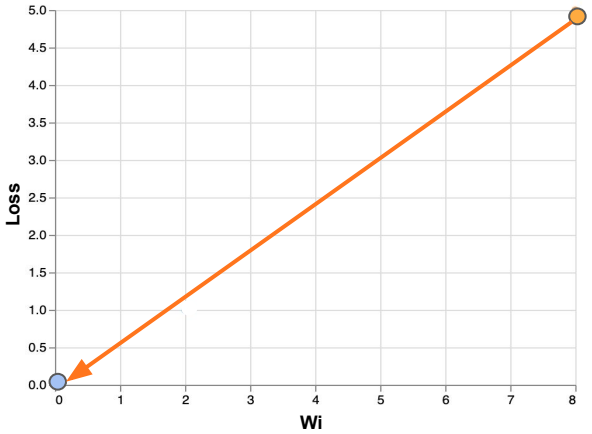
Parameters: **alpha : *float, default=1.0***

Constant that multiplies the L1 term. Defaults to 1.0. `alpha = 0` is equivalent to an ordinary least square, solved by the `LinearRegression` object. For numerical reasons, using `alpha = 0` with the `Lasso` object is not advised. Given this, you should use the `LinearRegression` object.

L2 Regularisation

The penalty component of L2 Regularisation is the squared value of the weights.

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} w_i^2$$



L2 Regularisation tries to reduce the weight values

Ridge Regression

Ridge Regression is the linear model that implements L2 Regularisation

```
from sklearn.linear_model import Ridge

ridge_reg = Ridge(alpha=0.1)
ridge_reg.fit(X, y)
```

Parameters: **alpha : *{float, ndarray of shape (n_targets,)}, default=1.0***

Regularization strength; must be a positive float. Regularization improves the conditioning of the problem and reduces the variance of the estimates. Larger values specify stronger regularization. Alpha

Elastic-Net Regression

Elastic-Net Regression is the linear model that implements both L1 and L2 Regularisations

```
from sklearn.linear_model import ElasticNet

elasticnet_reg = ElasticNet(alpha=0.1, l1_ratio=0.5)
elasticnet_reg.fit(X, y)
```

Parameters:

alpha : *float, default=1.0*

Constant that multiplies the penalty terms. Defaults to 1.0. See the notes for the exact mathematical meaning of this parameter. `alpha = 0` is equivalent to an ordinary least square, solved by the `LinearRegression` object. For numerical reasons, using `alpha = 0` with the `Lasso` object is not advised. Given this, you should use the `LinearRegression` object.

l1_ratio : *float, default=0.5*

The ElasticNet mixing parameter, with `0 <= l1_ratio <= 1`. For `l1_ratio = 0` the penalty is an L2 penalty. `For l1_ratio = 1` it is an L1 penalty. For `0 < l1_ratio < 1`, the penalty is a combination of L1 and L2.

Assignment

Assignment 1

Objective:

Your task is to train some regression models that accurately predict rental price, analyse their results, assess their benefits and risks according to a business use case of your choice. You will need to define who will be the users of your models, why they will use your predictions and what will be the impacts for them.

This assignment requires you to run multiple experiments with the following instructions:

- **Experiment 0**: Perform EDA, data preparation, pick any number of numerical and/or categorical features of your choice from the dataset, assess baseline model performance.
- **Experiment 1**: With the same chosen features as for experiment 1, train multivariate linear regression models with different values for the hyperperparameter: fit_intercept, analyse the results and make recommendations for next experiments.
- **Experiment 2**: With the same chosen features as for experiment 1, train ElasticNet regression models with different values for the hyperperparameters: alpha and l1_ratio, analyse the results and make recommendations for next experiments.
- **Experiment 3**: With the same chosen features as for experiment 1, train KNN regression models with different values for the hyperperparameter: n_neighbors and p, analyse the results and make recommendations for next experiments.

Assignment 1

Due Date: 29/03/2025 11:59pm

Submission Requirements:

You will need to submit the following files on Canvas:

- 4 Jupyter notebooks (one for each experiment) as **.ipynb** files
- 4 Jupyter notebooks (one for each experiment) as **.pdf** files
- 1 final report and submit it on Canvas as **.pdf** file
- 1 .zip file containing all your files

In total you should submit 10 files on Canvas.

Note: Be careful with the naming convention

More information can be found on Canvas:
https://canvas.uts.edu.au/courses/34235/assignments/209081

Assessment Criteria:

	Criteria	Weight
1.	Quality, relevance and cleanliness of code and visualisation	20%
2.	Quality of scientific experimentation and analytic investigation	20%
3.	Depth of discussion of ethics/privacy issues (including matters related to Indigenous people), value, benefits, risks and recommendation for business stakeholders and final users	17%
4.	Strength of justification and explanation for features selected and model used	25%
5.	Clarity and quality of written report, data visualisation and appropriateness of communication style to audience	18%

Time for Lab

33